DATA 260 Homework 9

Xuanhua Li
019122974

Model Name: MacBook Air

Model Identifier: Mac16,13

Chip: Apple M4

Memory: 16 GB

This homework extends the existing HW8 rental-listing agent system. The implementation uses MySQL as the relational source of truth, Redis 8.10.0 for counters and semantic caching, SentenceTransformers for query embeddings, Kafka for the HW8 Planner–Writer–Reviewer pipeline, and the local qwen3:8b model through Ollama.

All Redis keys use the assigned s2974 prefix.

Implementation and Evidence Commit: 9b5e2df57cf65328e405b6eb7dccf4ace74bc947

GitHub Repository: https://github.com/carrovo/data260-2974

Submission Branch: hw9-work

Submission Tag: hw9

Configuration	Value
SID4	2974
PORT_BASE	8274
PREFIX	s2974
SEED	2974
VERIFY_SEED	262974
DOMAIN_ID	6
Primary domain entity	Rental listing

Part 1: Redis-Backed View Counter

Implementation

I added a viewCount column to the existing listings table while preserving all 5,000 rental-listing records. The migration is idempotent and can be run more than once without creating a duplicate column.

The new increment_listing_views() method first verifies that the listing exists. It initializes a missing Redis counter from the persisted database value and uses Redis `INCR` for every access. Every tenth access is committed to MySQL.

```
key = f"{PREFIX}:views:listing:{listing_id}"

current_count = int(self.redis_client.incr(key))

if current_count % VIEW_SYNC_INTERVAL == 0:

    listing.viewCount = current_count

    self.db.commit()

    logger.info("Synced views to database")


return current_count
```

Result

After 25 simulated accesses, Redis contained a count of 25. MySQL contained 20 because persistence occurred at accesses 10 and 20, as required. This design reduces database writes while still periodically preserving the count.

```
' reports/hw09/raw/part1_counter_demo.txt
Student: Xuanhua Li
Listing ID: 4
Redis key: s2974:views:listing:4
=== Testing Counter ===
Access #1: Total = 1
Access #2: Total = 2
Access #3: Total = 3
Access #10: Total = 10
✓ Synced views to database
Access #11: Total = 11
Access #20: Total = 20
✓ Synced views to database
Access #21: Total = 21
Access #25: Total = 25
=== Final Verification ===
Redis count: 25
Database count: 20
PART1_STATUS: PASS
carol@Carols-MacBook-Air data260-2974-hw7 %
```

Figure 1. Redis view counter reaching 25 accesses and synchronizing with MySQL at accesses 10 and 20.

Part 2: Semantic Response Cache

Architecture

I placed a Redis semantic cache in front of the HW8 Q&A pipeline. Each cache entry stores:

- The original query
- Its SentenceTransformer embedding
- The final agent answer
- A creation timestamp
- A relational data-version marker

Redis vector search finds the closest stored query using cosine similarity. The initial threshold was 0.85.

On a cache hit, the handler returns the cached answer and bypasses the HW8 Planner–Writer–Reviewer pipeline. On a miss, it runs the original pipeline and stores the final reviewed answer.

```
lookup = await asyncio.to_thread(

    cache.find, normalized_question

)

if lookup.is_hit:

    return {

        "answer": lookup.answer,

        "is_cached": True,

        "similarity_score": lookup.similarity,

    }

answer = await pipeline_runner(normalized_question)

cache.store(normalized_question, answer)
```

The API response includes is_cached, similarity_score, latency_ms, and data_version.

Measured Results

I tested 10 queries containing exact duplicates, paraphrases, and completely new questions.

Metric	Observed result
Total queries	10
Cache hits	5
Cache misses	5
Cache hit rate	50.0%
Average cached latency	16.90 ms
Average non-cached latency	21,084.11 ms

The cached responses were substantially faster because they bypassed the local multi-agent pipeline. These are observed local measurements rather than a guaranteed speed-up.

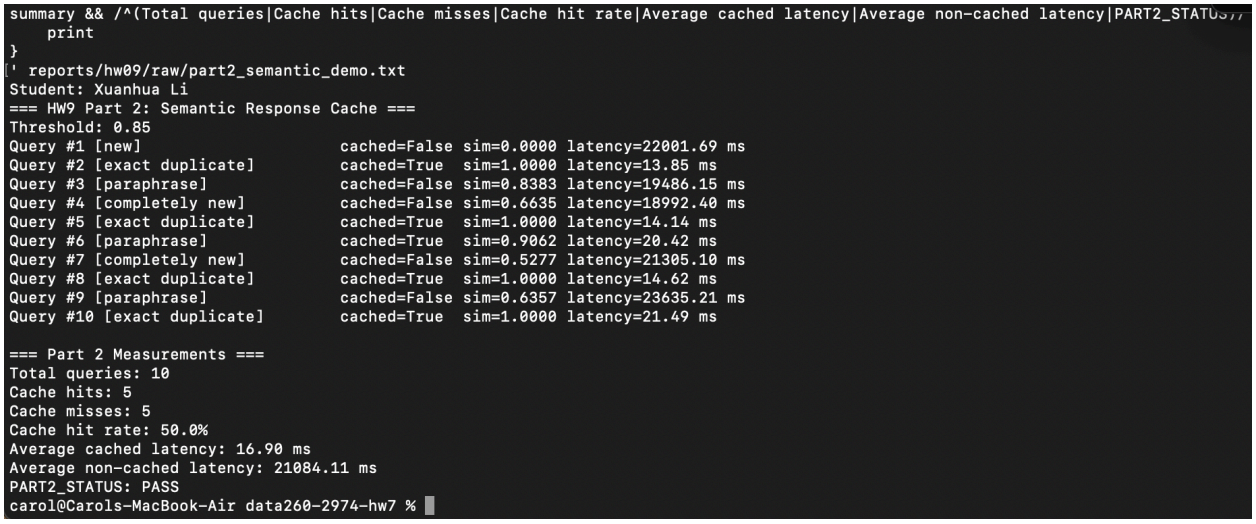


Figure 2. Semantic-cache experiment showing hits, misses, similarity scores, latency, and a 50.0% hit rate.

Part 3: Threshold Economics and Stale Answers

3.1 Frozen Labeled Dataset

Before performing the threshold sweep, I created and committed cache_cases.yaml with 40 labeled cases:

- 10 exact duplicates that should hit
- 10 paraphrases that should hit
- 10 similar questions with different correct answers that should miss
- 10 unrelated questions that should miss

The frozen file's SHA-256 digest was:
d890964a9274795d0afb18828776e13ed863c3e391024aebb6063750b6aea179

The labels were not modified after observing the threshold results.

3.2 Threshold Sweep

Each threshold was evaluated on the same 40 frozen cases. False-hit rate was calculated using the 20 cases labeled as "should miss."

Threshold	Hit rate	False-hit rate	Mean latency	Total cost
0.75	52.5%	25.0%	14.029 ms	$10.38
0.80	45.0%	20.0%	17.176 ms	$8.44
0.85	32.5%	10.0%	16.318 ms	$4.54
0.90	30.0%	5.0%	16.879 ms	$2.56
0.95	27.5%	5.0%	13.629 ms	$2.58

Higher thresholds reduced false hits but also rejected more valid paraphrases. The small latency differences between thresholds reflect local runtime variation; the important trade-off was between cache reuse and correctness.

```
print(f"Reason: {cost['selection_reason']}")
print("THRESHOLD_COST_STATUS: PASS")
[PY                                                                                                                    ]
Student: Xuanhua Li
=== HW9 Part 3: Threshold Sweep and Cost Selection ===
Threshold | Hit rate | False-hit rate | Mean latency | Total cost
0.75      |  52.5%   |   25.0%        |  14.029 ms   | $10.38
0.80      |  45.0%   |   20.0%        |  17.176 ms   | $8.44
0.85      |  32.5%   |   10.0%        |  16.318 ms   | $4.54
0.90      |  30.0%   |    5.0%        |  16.879 ms   | $2.56
0.95      |  27.5%   |    5.0%        |  13.629 ms   | $2.58

Selected threshold: 0.90
Reason: Threshold 0.90 has the lowest modeled total cost ($2.56). It reduces false hits to one while avoiding the addi
tional miss introduced at 0.95.
THRESHOLD_COST_STATUS: PASS
carol@Carols-MacBook-Air data260-2974-hw7 %
```

Figure 3. Threshold sweep, modeled cost, and selection of the 0.90 production threshold.

3.3 Complete False-Hit Example

A false hit remained even at the 0.95 threshold.

Cached query: What is the monthly rent for rental listing 4?

Incoming query: What is the monthly rent for rental listing 5?

Similarity score: 0.954607

Incorrect cached answer returned: Rental listing 4 (S2974-000004) has a monthly rent of $1,700.00.

Correct answer: Rental listing 5 (S2974-000005) has a monthly rent of $1,750.00.

The two questions have almost identical wording and intent. Only the numeric listing identifier changes. The embedding emphasized the shared “monthly rent” semantics but underweighted the record ID, even though that ID changes the correct database answer.

```
[carol@Carols-MacBook-Air data260-2974-hw7 % cat reports/hw09/raw/part3_false_hit.txt
Student: Xuanhua Li
=== HW9 Part 3.3: Complete False Hit ===
Case: similar_miss_02
Threshold: 0.95
Cached query: What is the monthly rent for rental listing 4?
Incoming query: What is the monthly rent for rental listing 5?
Similarity score: 0.954607
Cached answer returned: Rental listing 4 (S2974-000004) has a monthly rent of $1,700.00.
Correct answer: Rental listing 5 (S2974-000005) has a monthly rent of $1,750.00.
Why the embedding was confused: The two questions have nearly identical wording and intent; only the numeric entity id
entifier changes from 4 to 5. The embedding emphasizes the shared monthly-rent semantics and underweights the record I
D, even though that ID changes the correct database answer.
FALSE_HIT_STATUS: PASS
carol@Carols-MacBook-Air data260-2974-hw7 %
```

Figure 4. Complete false hit caused by two nearly identical questions referring to different listing IDs.

3.4 Cost Model and Threshold Choice

I assigned a cache-miss cost of $0.02 to represent local inference time, electricity, and system resources. I assigned a false-hit cost of $2.00 to represent the larger human-review or support cost caused by returning a plausible but incorrect domain answer.

The model was: Total cost = miss count × $0.02 + false-hit count × $2.00

Threshold 0.90 produced the lowest modeled cost at $2.56. It reduced false hits to one while avoiding the additional cache miss introduced at 0.95. Therefore, I selected 0.90 as the threshold to ship.

3.5 Stale Answers and Invalidation

I cached the rent for listing 4 when its MySQL value was $1,700.00. I then changed the database value to $1,725.00. Without invalidation, Redis returned the old $1,700.00 answer, reproducing a stale cache hit.

Each semantic-cache entry stores the relational data version from the time it was created. After a relevant relational write, the system increments the Redis data-version key. A version-aware lookup rejects and deletes an entry when its stored version differs from the current version.

```
if (
    enforce_data_version
    and cached_version != self.current_data_version()
):
    self.redis_client.delete(cache_key)
    return CacheLookup(
        None, similarity, matched_query,
        cached_version, cache_key
    )
```

After incrementing the data version from 1 to 2, the stale entry was rejected and the refreshed $1,725.00 answer was cached. The original database rent was restored to $1,700.00 after the demonstration, and the data version advanced to 3.

The advantage of this global-version strategy is simplicity and strong stale-answer protection. Its limitation is coarse invalidation: one relevant listing update can invalidate other older entries, lowering hit rate and causing additional recomputation.

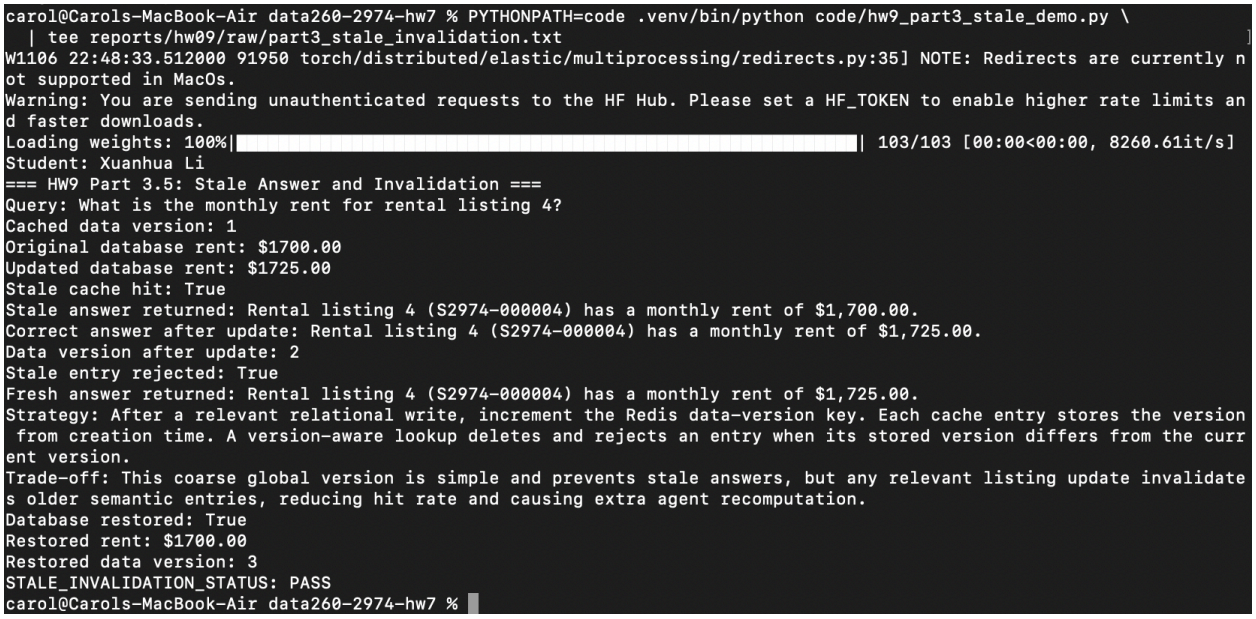


Figure 5. Stale answer reproduction, data-version invalidation, refreshed answer, and database restoration.

Part 4: Self-Improving Multi-Agent System

Dataset Separation

I created a separate reference dataset containing the ground-truth facts used by the Evaluator Agent. The three input splits contain only question_id and question; they do not expose expected answers to the Answering Agent.

Dataset	Number of questions	Purpose
training_data.json	3	Agent development and lesson creation
validation_data.json	4	Lesson acceptance and regression testing
test_data.json	4	Final held-out evaluation
reference_data.json	11 records	Ground-truth evaluation facts

The held-out test set was not used to create or revise lessons.

4.1 Evaluator Feedback and Repair

The system contains an Answering Agent and an Evaluator Agent. The Evaluator compares each answer with the matching reference record and returns either PASS or REVISE, together with an explanation and correction feedback.

The controlled initial answer intentionally omitted the exact rent and listing code. The Evaluator returned REVISE and supplied the missing reference facts. The Answering Agent then produced:

The monthly rent for rental listing 4 is $1,700.00, and its listing code is S2974-000004.

The revised answer passed on attempt 2. The measured sequence latency was 17,115.49 ms.

The coordinator enforces a maximum of two answer attempts:

```
for attempt_number in (1, 2):

    answer = answering_agent.answer(

        question=question,

        feedback=feedback,

        lessons=lessons,

    )
```

```
    evaluation = evaluator_agent.evaluate(
        question_id=question_id,
        question=question,
        answer=answer,
    )
    if evaluation.result == "PASS":
        break
    if not allow_revision or attempt_number == 2:
        break
    feedback = evaluation.feedback
```

```
Student: Xuanhua Li
=== HW9 Part 4.1: Complete Revision Sequence ===
Question: What is the monthly rent and listing code for rental listing 4?
Initial answer: The listing has a recorded monthly rent and listing code, but those exact values should be verified be
fore replying.
Initial evaluation: REVISE
Evaluator feedback: Include these ground-truth facts: Rental listing 4 has listing code S2974-000004. Its monthly rent
 is $1,700.00.
Revised answer: The monthly rent for rental listing 4 is $1,700.00, and its listing code is S2974-000004.
Final evaluation: PASS
Number of attempts: 2
Final response: The monthly rent for rental listing 4 is $1,700.00, and its listing code is S2974-000004.
Latency: 17115.49 ms
PART4_1_STATUS: PASS
carol@Carols-MacBook-Air data260-2974-hw7 %
```

Figure 6. Complete Answering Agent–Evaluator Agent revision sequence ending in PASS after two attempts.

4.2 Saving and Reusing a Lesson

The repeated failure pattern was that listing-specific answers omitted exact listing codes or current numeric values. I stored the following reusable lesson in lessons.json:

- Lesson ID: lesson_001
- Failure pattern: Listing-specific answers omit the exact current value or listing code.
- Correction: Load the matching reference record and include the exact listing code and numeric value.
- Created at: 2026-11-07
- Use count: 1

For a later question about listing 5, the system matched the lesson using its keywords, loaded it before answering, and returned the correct code S2974-000005 and rent $1,750.00. The answer passed on the first attempt.

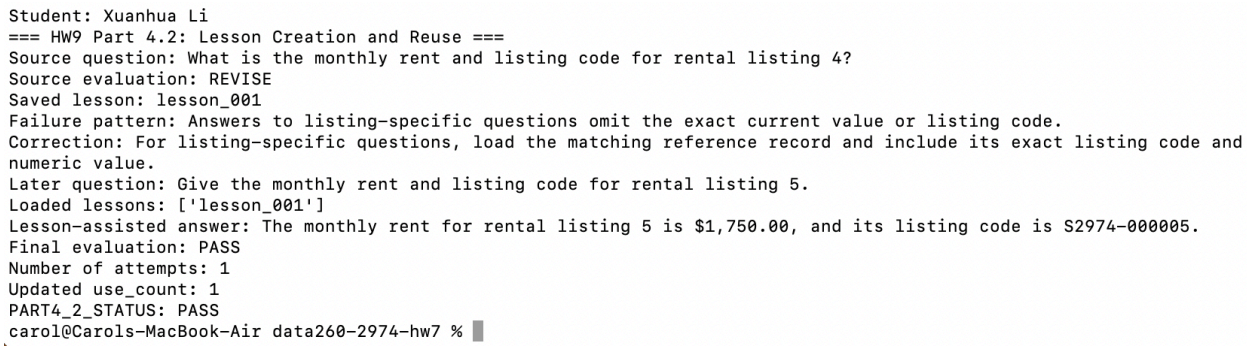


Figure 7. lesson_001 being loaded and reused for a later related question.

4.3 Validation Before Keeping the Lesson

The candidate lesson was evaluated on validation_data.json before it was retained.

Case	Before lesson	After lesson	Lesson loaded after
validation_01	REVISE	PASS	lesson_001
validation_02	REVISE	PASS	lesson_001
validation_03	REVISE	REVISE	None
validation_04	PASS	PASS	None

Correct answers improved from 1/4 to 3/4. The previously correct unrelated answer remained correct, and unrelated questions did not receive unsupported listing facts. The test dataset was not used during validation. Because the lesson improved the relevant cases without causing a regression, the validation gate kept lesson_001.

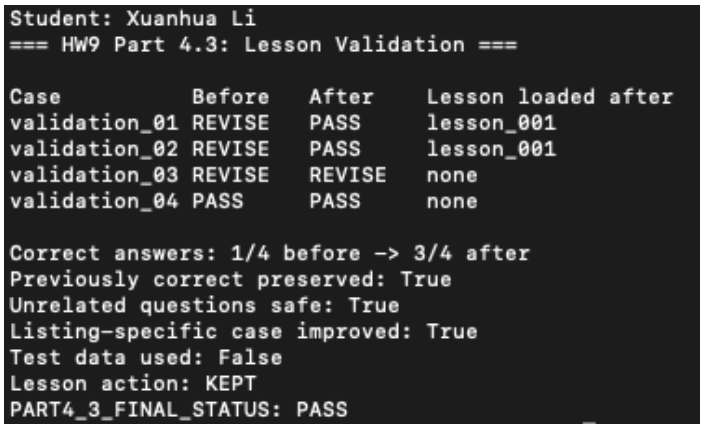


Figure 8. Before-and-after validation demonstrating improvement, preservation, and the decision to keep the lesson.

4.4 Four System Variants

All four variants were independently evaluated on the same four held-out test questions. The test dataset's SHA-256 digest was checked before and after evaluation and remained unchanged.

Variant	Correct	Incorrect	Average attempts	Average latency
Baseline	0	4	1.00	9,351.00 ms
Evaluator Repair	4	0	2.00	11,407.78 ms
Lesson Memory	4	0	1.25	7,167.10 ms
Validated Improvement	4	0	1.25	6,703.90 ms

The Baseline did not have access to evaluator feedback or reference-backed lessons and failed all four cases. Evaluator Repair corrected all four answers, but it required two attempts per question and had the highest observed latency.

Lesson Memory and Validated Improvement both achieved 4/4 correct answers with an average of 1.25 attempts. Their identical correctness is expected because they use the same lesson. The important distinction is that Validated Improvement only loads the lesson after the lesson passes the validation gate.

The observed latency of Validated Improvement was the lowest in this run, but local-model latency can vary with system load. The strongest conclusion is that reusable lessons reduced the number of repeated revisions while preserving correctness.

```
Student: Xuanhua Li
=== HW9 Part 4.4: Four System Variants ===

Variant              Correct  Incorrect  Avg attempts  Avg latency
Baseline                   0          4          1.00      9351.00 ms
Evaluator Repair           4          0          2.00     11407.78 ms
Lesson Memory              4          0          1.25      7167.10 ms
Validated Improvement      4          0          1.25      6703.90 ms

Held-out test cases: 4
Test dataset unchanged: True
Maximum answer attempts: 2
Lessons created from test data: False
Validated lesson loaded: lesson_001
PART4_4_FINAL_STATUS: PASS
```

Figure 9. Final comparison of Baseline, Evaluator Repair, Lesson Memory, and Validated Improvement.

Agent Communication, Stopping, and Limitation

The Answering Agent produces an initial response. The Evaluator Agent checks that response against the separate reference dataset. If revision is required, its feedback is returned to the Answering Agent for one final attempt. The system stops immediately after a PASS or after the second attempt.

Before each answer, the system searches lessons.json for lessons whose keywords match the question. Relevant corrections and reference facts are then included in the Answering Agent's context.

One limitation is that lesson retrieval currently relies on keyword matching. A relevant lesson may be missed when a question uses different vocabulary, and a broad keyword may load a lesson unnecessarily. The evaluation dataset is also intentionally small, so the measured results should not be interpreted as proof of general performance across all rental questions.

Testing and Reproducibility

The implementation includes offline tests for:

- Redis view-counter synchronization
- Cache-hit and cache-miss routing
- Threshold and cost calculations
- Maximum two-attempt enforcement
- Lesson loading and use-count persistence
- Validation-gate behavior
- Four-variant metric aggregation

Ruff checks, Python compilation, offline tests, Docker Compose validation, Redis health checks, database restoration checks, and Git whitespace checks all passed. The final implementation is committed to the hw9-work branch.

AI Use

ChatGPT was used to help interpret the assignment requirements, organize the implementation plan, troubleshoot errors, and review code structure. I ran the commands locally, reviewed and modified code, verified Redis and MySQL state, executed the tests and experiments, inspected the measured outputs, selected the final threshold, and confirmed the final repository contents before submission.

Conclusion

This homework added two levels of system improvement to the rental-listing application. Redis reduced repeated database and agent work through counters and semantic response caching. The threshold experiment showed that higher cache reuse is not always desirable because false hits can return plausible but incorrect answers.

The self-improving agent workflow then demonstrated how evaluator feedback, reusable lessons, and validation gates can improve answer correctness. The final validated system achieved 4/4 correct held-out answers while averaging 1.25 attempts per question.